Deploy on Arc

Copy page

Learn how to deploy, test, and interact with a Solidity smart contract on the Arc Testnet.

> Arc is currently in its testnet phase. During this period, the network may experience instability or unplanned downtime. **Note:** Throughout this page, all references to Arc refer specifically to the Arc Testnet.

In this tutorial, you'll use Solidity and Foundry to write, deploy, and interact with a simple smart contract on the Arc Testnet.

What you'll learn

By the end of this tutorial, you'll be able to:

- Set up your development environment
- Configure Foundry to connect with Arc
- Implement your smart contract
- Deploy your contract to Arc Testnet
- Interact with your deployed contract

Set up your development environment

Before you deploy to Arc, you need a working development environment. In this step, you install **Foundry**, a portable Ethereum development toolkit, and initialize a new Solidity project.

1. Install Development Tools

```
# Download foundry installer `foundryup`
curl -L https://foundry.paradigm.xyz | bash
```

2. Install binaries

```
# Install forge, cast, anvil, chisel
foundryup
```

3. Initialize a new Solidity Project

```
forge init hello-arc && cd hello-arc
```

Configure Foundry to interact with Arc

In this step, you set up Foundry to connect to the Arc network by adding Arc's RPC URLs to your project environment.

1. Create a `.env` file

Open the `hello-arc` project in your preferred code editor (for example, **VS Code**). Then, create a new file named `.env` in the root of the project directory.

2. Add the Arc Testnet RPC URL

Paste the following environment variable into the `.env` file:

```
ARC_TESTNET_RPC_URL="https://rpc.testnet.arc.network"
```

This URL allows Foundry to connect to the Arc Testnet.

> Never commit your `.env` file to version control. Store private keys and sensitive variables securely.

Implement your smart contract

In this step, you create the **HelloArchitect** contract, update the test and script files, and compile the project.

HelloArchitect is a simple storage contract that manages a greeting message: it starts with a default greeting, lets you update it, and emits an event whenever the greeting changes.

1. Write the HelloArchitect contract

First, delete the default `Counter.sol` template file from the `/src` directory:

```
rm src/Counter.sol
```

Next, create a new file named `HelloArchitect.sol` inside the `/src` directory, and add the following code:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.30;

contract HelloArchitect {
    string private greeting;

    // Event emitted when the greeting is changed
    event GreetingChanged(string newGreeting);

    // Constructor that sets the initial greeting to "Hello Architect!"
    constructor() {
        greeting = "Hello Architect!";
    }

    // Setter function to update the greeting
    function setGreeting(string memory newGreeting) public {
        greeting = newGreeting;
        emit GreetingChanged(newGreeting);
    }

    // Getter function to return the current greeting
    function getGreeting() public view returns (string memory) {
        return greeting;
    }
}
```

This contract includes a private `greeting` variable that stores the greeting string, along with two public functions:

- `setGreeting` updates the `greeting` value and emits the `GreetingChanged` event
- `getGreeting` returns the current value of `greeting`

2. Update scripts and tests

Since you deleted `Counter.sol`, you need to remove or replace any scripts and tests that reference it to avoid compilation errors.

Delete the `script` directory

The `script` directory includes files that reference `Counter.sol`. Since you've removed `Counter.sol`, delete the entire `script` directory to avoid compilation errors:

```
rm -rf script
```

You can recreate this directory later with updated deployment scripts for your own contracts.

Replace `Counter.t.sol` with `HelloArchitect.t.sol`

Navigate to the `/test` directory, delete the existing `Counter.t.sol` file, and create a new test file named `HelloArchitect.t.sol`. Then, add the following test cases to validate your contract:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.30;

import "forge-std/Test.sol";
import "../src/HelloArchitect.sol";

contract HelloArchitectTest is Test {
    HelloArchitect helloArchitect;

    function setUp() public {
        helloArchitect = new HelloArchitect();
    }

    function testInitialGreeting() public view {
        string memory expected = "Hello Architect!";
        string memory actual = helloArchitect.getGreeting();
        assertEq(actual, expected);
    }

    function testSetGreeting() public {
        string memory newGreeting = "Welcome to Arc Chain!";
        helloArchitect.setGreeting(newGreeting);
        string memory actual = helloArchitect.getGreeting();
        assertEq(actual, newGreeting);
    }

    function testGreetingChangedEvent() public {
        string memory newGreeting = "Building on Arc!";

        // Expect the GreetingChanged event to be emitted
        vm.expectEmit(true, true, true, true);
        emit HelloArchitect.GreetingChanged(newGreeting);

        helloArchitect.setGreeting(newGreeting);
    }
}
```

3. Test the contract

Run the following command to execute the contract's unit tests locally:

```
forge test
```

This will compile the project, run the tests defined in `HelloArchitect.t.sol`, and display the results in your terminal.

4. Compile the contract

To compile the **HelloArchitect** contract and generate build artifacts, run:

```
forge build
```

This creates the `/out` directory containing the compiled bytecode and ABI, which you'll use when deploying the contract.

Deploy your contract to Arc testnet

In this step, you generate a wallet, fund it with testnet USDC (Arc's native gas token), and deploy your smart contract to the Arc Testnet using Foundry.

1. Generate a wallet

To deploy the **HelloArchitect** contract, you need a funded wallet. Use the Foundry command-line tool to generate a new wallet:

```
cast wallet new
```

The command generates a new keypair and returns output similar to the following:

```
Successfully created new keypair.
Address:     0xB815A0c4bC23930119324d4359dB65e27A846A2d
Private key: 0xcc1b30a6af68ea9a9917f1dd•••••••••••••••••••••••••••••••••••••97
```

Important: Keep your private key secure. Never share it or commit it to source control.

Add your private key to your `.env` file:

```
PRIVATE_KEY="0x..."
```

Reload your environment variables:

```
source .env
```

2. Fund your wallet

Visit the **Circle Faucet**, select **Arc Testnet**, paste your wallet address, and request testnet USDC.

Since USDC is Arc's native gas token, this will provide the funds needed to cover gas fees when deploying your contract.

Testnet USDC is for testing purposes only. It has no real-world value and must not be used in production.

3. Deploy the contract

With your wallet funded with testnet USDC, deploy the **HelloArchitect** contract to the Arc Testnet using the Foundry command-line tool:

```
forge create src/HelloArchitect.sol:HelloArchitect \
  --rpc-url $ARC_TESTNET_RPC_URL \
  --private-key $PRIVATE_KEY \
  --broadcast
```

Important: Never expose your real private key in production. Use environment variables or secrets management in real deployments.

After the contract is deployed successfully, you should see output similar to this:

```
Compiler run successful!
Deployer: 0xB815A0c4bC23930119324d4359dB65e27A846A2d
Deployed to: 0x32368037b14819C9e5Dbe96b3d67C59b8c65c4BF
Transaction hash: 0xeba0fcb5e528d586db0aeb2465a8fad0299330a9773ca62818a1827560a
```

4. Store the contract address

Copy the deployed contract address from the `Deployed to:` line and save it to your `.env` file:

```
HELLOARCHITECT_ADDRESS="0x..."
```

Reload your environment variables again:

```
source .env
```

Interact with your deployed contract

In this step, you verify that the deployment succeeded by checking the transaction in the Arc Testnet Explorer, then use `cast` to call a function from your contract.

1. Check transaction on the explorer

Open the **Arc Testnet Explorer**, and paste the **transaction hash** from the deployment output. This lets you view the transaction details and confirm that the contract was deployed successfully.

2. Use `cast` to call a contract function

Use the `cast call` command to interact with your deployed contract from the command line. Run the following:

```
cast call $HELLOARCHITECT_ADDRESS "getGreeting()(string)" \
  --rpc-url $ARC_TESTNET_RPC_URL
```

The command calls the `getGreeting` function on the **HelloArchitect** contract and returns the current value of the `greeting` variable.

Next steps

Congratulations, you've deployed and interacted with your first contract on Arc Testnet. From here, you can:

- Extend the **HelloArchitect** contract with more logic for additional features.
- Explore Arc's stablecoin-native features like USDC as gas and deterministic finality
- Build more advanced applications for payments, FX, or tokenized assets

Was this page helpful? Yes No

‹ Connect to Arc MCP server ›

Links
Network status
Blog
Community
Arc homepage

Resources
Terms of service
Privacy policy
Cookie policy

Powered by mintlify

By using the Arc Network Testnet, you agree to the Terms of Service and Privacy Policy.